

LEGALEASE

5. Project Development Phase

LegalEase – Implementation and Integration

PROJECT TEAM

- Nikkath Khatoon (Team Leader)
- Nowfel.M
- Snehan.P
- Joel.J
- Arfana Begum

1. Development Overview

Prepared using the supplied Periyar University-style reference format and the supplied LegalEase project document. The reference structure covers project title/description, use cases, architecture, prerequisites, objectives, workflow, milestones, testing, deployment, results, limitations, future enhancements and appendix. The LegalEase source describes an AI-powered legal document generator using Streamlit, FastAPI, Gemini integration and TXT/DOCX/PDF export.

2. Environment Setup

The supplied LegalEase documentation lists Python 3.10+, FastAPI, Uvicorn, Streamlit, Google Generative AI SDK, python-docx, FPDF, Pillow, requests and python-dotenv.

3. Backend Development

- Initialize FastAPI
- Create root health endpoint
- Include routes
- Define DocumentRequest
- Implement POST /generate
- Call GeminiDocumentGenerator

4. AI Integration

GeminiDocumentGenerator forms a structured prompt, sends it to the Gemini service and returns generated text to FastAPI.

5. Frontend Development

- Streamlit page configuration
- Input fields
- Generate action
- Preview
- Editable text area
- TXT/DOCX/PDF downloads

6. Formatting Modules

- sanitize_text
- format_docx
- format_pdf
- format_html_preview

7. Integration

The development integrates Streamlit, FastAPI, Gemini generation and output formatting into one end-to-end workflow.

8. Development Result

The result is an academic prototype capable of accepting legal-document inputs, generating AI-assisted content, presenting an editable preview and exporting the result.